Road Crack Anomaly Detection on SDNET2018

Course	50.039 Theory and Practice of Deep Learning
Instructors	Prof. Matthieu De Mari & Prof. Dileepa Fernando
Group	35

Group Members & Contributions

Name	Student ID	Primary Contribution	Secondary Contribution
Avitra Phon	1006946	Deep SVDD architecture, ResNet-18 encoder, ProtoNet, LIME /CLAHE preprocessing pipeline, Student-Teacher	Dataset patch extraction, evaluation metrics, report writing
Marcus Lim	1006855	PatchCore, Autoencoder, Binary CNN, ProtoNet ablation & final model, visualisations	Hyperparameter experiments, SOTA comparison, report writing

---

How to Run This Project

Git clone the repository:

https://github.com/Marcooos05/sdnet2018-crack-anomaly-detection.git

Environment Setup

pip install torch torchvision numpy matplotlib scikit-learn pillow opencv-python

Full dependencies: *torch, torchvision, numpy, matplotlib, scikit-learn, Pillow, opencv-python*

In a venv run `` pip install -r requirements.txt `` from the root of the repository

Dataset

Download SDNET2018 and place it in the project root as *SDNET2018/*.
Source: https://digitalcommons.usu.edu/all_datasets/48/

Train All Models from Scratch

Run notebooks in this order:

Step	Notebook	Description
1	*0_data_preprocessing.ipynb*	Build stratified train/val/test splits
2	*1_deep_svdd_with_F_norm.ipynb*	Deep SVDD (all 4 iterations)
3	*2_autoencoder.ipynb*	ConvAE (all 5 variants)
4	*7_patch_svdd.ipynb*	Patch-level SVDD
5	*2_patchcore_efficientnet.ipynb*	PatchCore EfficientNet-B4 (training-free)
6	*8_student_teacher.ipynb*	Student-Teacher (all 5 variants)
7	*6_binary_cnn.ipynb*	Supervised upper-bound baseline
8	*4_few_shot.ipynb*	ProtoNet baseline
9	*9_protonet_ablation.ipynb*	ProtoNet hyperparameter ablation
10	*10_protonet_final.ipynb*	**Final best model**

Reproduce Results Without Retraining

1. Ensure *checkpoints/proto_final.pt* is present
2. Open *10_protonet_final.ipynb*
3. Run cells 1–5 (imports, data splits, dataset, model definition, helpers)
4. Skip to **Section 7 (Evaluate on Test Set)** — loads the checkpoint and reproduces all metrics and plots without retraining

---

1. Introduction & Problem Statement

1.1 Overview

Road infrastructure deterioration — cracking, potholing, and surface deformation — poses significant risks to public safety and incurs substantial maintenance costs when left undetected. Manual inspection is expensive, infrequent, and inconsistent. Deep learning-based automated detection offers a scalable, cost-effective alternative.

We frame this as a **patch-level anomaly detection problem**. Rather than treating cracks as an object detection task requiring bounding box annotations, we ask a simpler question: is this image patch a normal concrete surface, or does it contain a crack? The model outputs a scalar anomaly score; patches exceeding a calibrated threshold are flagged for maintenance.

This framing satisfies the anomaly detection requirement naturally, most pavement images are uncracked, producing the required class imbalance (approximately 89% normal, 11% cracked).

1.2 Project Evolution

The project initially aimed to use the SVRDD (2024), RDD2020, and RDD2022 road datasets with a 128×128 sliding-window patch extraction approach. After constructing the dataset, a critical issue emerged: despite restricting patches to road regions, many normal patches contained irrelevant objects (cars, motorcycles, lane markings) producing high intra-class variance. This is fatal for one-class learning where the model must learn what "normal" looks like, but if normal patches look dramatically different from each other, the hypersphere/prototype must be very large, making anomalies indistinguishable from the outlying normal patches.

We pivoted to the **SDNET2018** dataset, which provides close-up, controlled imagery of concrete surfaces with no confounding objects.

A second dataset issue emerged within SDNET2018 itself: the dataset contains three surface types — Bridge Decks (B), Pavements (P), and Walls (W) — with dramatically different textures. Mixing them would allow the model to achieve above-random performance by separating surface textures that happen to correlate with crack labels, not by genuinely detecting cracks. We restricted all experiments to **Pavement (P) surfaces only** to ensure the model learns crack-specific features.

---

2. Dataset Description

2.1 SDNET2018 - Pavement Subset

Split	Total	Normal (label=0)	Cracked (label=1)	Anomaly Rate
Train	17,033	15,207 (89.3%)	1,826 (10.7%)	10.7%
Val	3,650	3,259 (89.3%)	391 (10.7%)	10.7%
Test	3,651	3,260 (89.3%)	391 (10.7%)	10.7%
Total	**24,334**	**21,726**	**2,608**	**10.7%**

Splits are stratified to preserve the class ratio. The 8.3× normal-to-anomaly ratio satisfies the project's class imbalance requirement.

2.2 Inputs and Outputs

- **Input:** 256×256-pixel RGB images of concrete pavement surfaces
- **Output:** Scalar anomaly score, higher = more likely to contain a crack
- **Decision:** Score ≥ threshold τ → flagged as anomalous. τ is calibrated on the validation set by maximising F1 score.

2.3 Preprocessing

We evaluated three preprocessing pipelines, motivated by Freitas et al. (2025) who showed that preprocessing significantly affects road defect detection performance:

Method	Description	Motivation
None	Raw ImageNet normalisation only	Baseline
CLAHE	Contrast-Limited Adaptive Histogram Equalisation	Enhances local contrast, improves crack visibility in uniform surfaces
LIME	Low-Light Image Enhancement	Brightens shadow regions, recovers detail in underexposed patches

CLAHE was selected as the default for ProtoNet based on ablation results (Section 5). Interestingly, PatchCore with EfficientNet-B4 performs better without preprocessing — the backbone's richer features already capture contrast variations that CLAHE was compensating for in weaker encoders.

2.4 Evaluation Metrics

Given the class imbalance, all models are evaluated on:

- **AUROC**: primary metric; threshold-independent, measures ranking quality
- **AUPRC**: captures precision-recall behaviour under imbalance; a random classifier achieves AUPRC ≈ 0.107
- **F1** at threshold τ calibrated on the validation set
- **Confusion matrix** (TN, FP, FN, TP) on the held-out test set

---

3. Methodology & Model Progression

The project followed a logic-driven progression through architectural paradigms, each transition directly motivated by the observed failure mode of the previous method:

Deep SVDD (collapse) → ConvAE (over-recon) → Patch SVDD (collapse) → PatchCore (training-free SOTA, 0.77) → Student-Teacher (feature comparison, 0.77) → ProtoNet (best result 0.80 AUROC)

---

4. Deep SVDD

4.1 Concept

Deep SVDD (Ruff et al., 2018) maps all normal images into a compact hypersphere in embedding space. Anomalies, unseen during training, fall outside it. The anomaly score is the squared distance from the embedding to the learned centre **c**:

$$L = \frac{1}{n}\sum||\phi(x) - c||^2$$

4.2 Architecture

- Backbone: ResNet-18 (ImageNet pretrained, layers 3–4 fine-tuned, layers 1–2 frozen)
- Projection head: Linear (512→256) → Linear (256→64), **bias=False** throughout
- Training data: Normal images only (15,207 patches)

4.3 Iteration 1 — With F.normalize

Hypothesis: L2-normalising embeddings onto the unit sphere prevents scale collapse (where the network shrinks all weights to zero).

Result: AUROC 0.5781 - near random.

Why it failed: Once all embeddings are constrained to the unit sphere ($\|z\|=1$), the squared distance $\|z-c\|^2$ is bounded between 0 and 4 for all inputs, both normal and anomalous. The model has no geometric freedom to push crack embeddings further from **c** than normal embeddings. F.normalize hid the collapse by bounding the loss range, making it look like healthy training. Additionally, BatchNorm layers in the frozen ResNet-18 continued updating running statistics during SVDD training, shifting the embedding distribution toward zero-mean.

4.4 Iteration 2 - No Bias, No Normalisation, Frozen BatchNorm

Changes: Removed F.normalize; bias=False in all Linear layers; explicitly called *m.eval()* on all BatchNorm2d layers in the encoder to freeze running statistics.

Result: Best val AUROC 0.6703 - meaningful improvement but rapid loss collapse (epoch 1 → epoch 5 still shows sudden drop).

Analysis: Removing normalisation restored geometric freedom, the centre norm (2.44) is well above 1.0, confirming embeddings are no longer sphere constrained. However, the loss still collapses quickly. Global average pooling in ResNet-18 collapses the 8x8 spatial feature map into a single vector; a crack covering 3-5% of pixels contribute proportionally little to this average.

4.5 Iteration 3 - Adaptive MaxPool

Hypothesis: Max-pooling preserves peak activations from crack-containing regions rather than averaging them out.

Result: Best val AUROC 0.6432 - worse than AvgPool.

Why it failed: By layer 4, each of the 8x8 spatial cells already has a receptive field of ~180x180 pixels. A crack spanning 3% of the image may not dominate any single cell's activation. The limitation is not the pooling strategy but the depth at which spatial information is lost.

4.6 Iteration 4a - AutoEncoder Warm-Start

Hypothesis: Pre-training the encoder as an autoencoder (as recommended by Ruff et al., 2018) provides better initialisation and reduces early collapse.

Result: Best val AUROC 0.6540, AUROC 0.6434, AUPRC 0.1855, F1 0.2542.

Why it failed: Unlike Ruff et al. (2018), who trained from scratch, our encoder begins from ImageNet features. The additional 5 epochs of reconstruction pretraining add marginal information. Loss collapsed from 0.030 to 0.00006 in epoch 1→2 despite the warm start.

4.7 Iteration 4b - Squeeze-and-Excitation Attention

Hypothesis: SE attention re-weights each of 512 encoder channels, amplifying crack-relevant ones and suppressing background texture.

Result: AUROC 0.6153 - worse than baseline.

Why it failed: SE collapsed before it could learn meaningful channel weights, SVDD loss was near zero by epoch 2. Additionally, cracks are spatially localised (3-5% of pixels) but SE reweights channels globally across the entire image. A channel that activates on crack edges also activates on non-crack texture edges elsewhere, preventing clean suppression.

4.8 Deep SVDD Summary

Variant	Val AUROC	Test AUROC
+ F.normalize	-	0.5781
No bias/norm, frozen BN	0.6703	~0.64
+ AdaptiveMaxPool	0.6432	~0.64
+ AE warm-start	0.6540	0.6434
+ SE attention	0.6431	0.6153

Root cause diagnosis: Even after extensive ablation (spread loss weights, centre momentum, freeze depth, head architecture, bottleneck dimensions, LR, scheduler, preprocessing), the best val AUROC was 0.7138 but the test AUROC dropped back to 0.635, the model overfits to hyperparameter choices rather than genuine generalisation. The training loss collapses quickly yet AUROC barely moves: the hypersphere gets smaller without features becoming more discriminative.

SVDD works well on datasets like MVTec AD where normal samples have controlled intra-class variation and anomalies are visually distinct. On SDNET2018 pavements, normal images look dramatically different (wet, dry, shaded, rough) hence the hypersphere must be large to contain this variation, and a large hypersphere cannot discriminate against defects that are perceptually similar to rough-but-normal texture.

---

5. Convolutional Autoencoder

5.1 Concept

A symmetric encoder-decoder trained on normal images only. Anomalies are detected by high reconstruction error. A cracked patch, being out-of-distribution, is reconstructed poorly, producing higher MSE than a normal patch.

The reconstruction objective gives the encoder a stable gradient at every epoch, unlike SVDD's projection head which collapses within 1–2 epochs because there is nothing preventing it from mapping everything to a single point.

5.2 Architecture (Shared Across All Variants)

- **Encoder:** 4 conv blocks: Conv2d → BatchNorm → ReLU → MaxPool2d (2), channels 3→32→64→128→256
- **Bottleneck:** FC → 256-dim vector
- **Decoder:** 4 transpose conv blocks mirroring the encoder, channels 256→128→64→32→3, Sigmoid on final layer
- No skip connections (unlike U-Net) - the bottleneck is the only information pathway, preventing anomalies from being copied across directly
- **Anomaly score:** Pixel-wise MSE between input and reconstruction, averaged over spatial dimensions

5.3 Variants and Results

Variant	Key Addition	Val AUROC	Test AUROC	AUPRC	F1
A, Baseline	MSE loss	0.6142	0.6142	0.1497	0.2266
B, +SE	Channel attention (r=16)	0.6090	0.6087	0.1457	0.2279
C, +CBAM	Channel + spatial attention	0.6162	0.6152	0.1501	0.2254
D, +Perceptual	0.5×MSE + 0.5×VGG relu2_2	0.6140	0.6125	0.1490	-
E, ResNet18 enc	Frozen pretrained encoder	<0.60	0.6125	0.1490	-

Key observations:

SE (Variant B): Slightly hurts AUROC but shifts toward higher recall (222 TP vs 165) at the cost of far more false positives (1,335 FP vs 900). SE identifies some crack-relevant channels but simultaneously flags too much normal texture.

CBAM (Variant C): Marginally best AUROC (0.6152). Despite spatial attention being theoretically motivated, cracks are localised, the improvement is negligible and AUROC peaks at epoch 1 then immediately declines, suggesting CBAM fits spurious spatial patterns early then overfits.

Perceptual loss (Variant D): No meaningful difference. VGG relu2_2 features, trained on ImageNet object classification, are not sensitive enough to the subtle texture difference between concrete cracks and rough-but-normal concrete.

Pretrained ResNet18 encoder (Variant E): Counterintuitively worse. ResNet-18 downsamples aggressively via maxpool and strided convolutions, discarding low-level texture detail needed for meaningful reconstruction error. The decoder produces blurry average reconstructions for everything, normal and cracked alike.

5.4 Autoencoder Root Cause

All five variants cluster within a 0.59-0.62 AUROC band regardless of architectural changes. The root cause is the **reconstruction completeness problem** given sufficient capacity, the AE learns to reconstruct anomalies as well as normal samples, collapsing the anomaly score gap. This is evident in training dynamics where AUROC peaks within epochs 1-3 and either stagnates or declines as training continues, while the loss continues to decrease. The model gets better at reconstruction but worse at anomaly detection.

---

6. Patch SVDD

6.1 Concept

Standard image-level SVDD encodes an entire 256×256 image into a single embedding vector, heavily diluting the crack signal (which covers <5% of the image). Following Yi & Yoon (ACCV 2021), Patch SVDD divides each image into overlapping 64×64 patches (stride 32, ~49 patches per image). Each patch receives an individual SVDD anomaly score; the image-level score is the maximum patch score. This max-aggregation means a single anomalous patch is sufficient to flag the image.

6.2 Architecture and Results

Variant	Backbone	Params	AUROC	AUPRC	F1	TN	FP	FN	TP
Scratch CNN	3 conv blocks (3→32→64→128) + GAP	~110K	0.6853	0.1936	0.2936	2,628	632	215	176
Pretrained ResNet-18	Frozen layers 1-2, tuned 3-4	11.24M	0.7067	0.2178	0.3127	2,399	861	159	232

- Loss: SVDD – λ x spread regularisation (λ=1.0) to prevent collapse
- Adam, CosineAnnealingLR, 30 epochs, batch size 256 patches

Key finding: Both models exhibit the same failure mode where the best validation AUROC is always achieved at **epoch 1** (0.7579 scratch, 0.7280 pretrained), after which performance degrades. The SVDD loss rapidly collapses patch embeddings toward the centre in early epochs; the initial spread from a freshly initialised encoder provides the best discrimination, which training then destroys. Even a frozen ResNet-18 backbone cannot prevent this: the 512→128 projection head alone is sufficient to collapse the 512-dim ImageNet features into a degenerate subspace.

Patch granularity alone does not resolve the fundamental collapse problem. SDNET2018 cracks represent subtle texture changes rather than structurally distinct objects, patch-level and image-level representations are similarly ambiguous to a one-class detector.

---

7. PatchCore (SOTA Benchmark)

7.1 Concept

PatchCore (Roth et al., 2022) requires no training. It operates in two stages:

1. **Memory bank construction:** Extract spatial patch features from normal training images using a frozen pretrained backbone, then apply coreset subsampling (keeping 5%) to produce a compact set of representative normal patch descriptors.
2. **Inference:** For each test image, compute the nearest-neighbour distance to the memory bank per patch. The maximum patch distance is the image-level anomaly score.

The key insight: rather than asking "can we reconstruct this patch?" (which fails when the AE learns to reconstruct anomalies), PatchCore asks "have we seen a patch like this before?" A crack patch simply has no close neighbour in a bank built entirely from normal patches.

7.2 Architecture

- **Backbone:** EfficientNet-B4 (pretrained, fully frozen), chosen over ResNet-18 for richer spatial features via compound scaling
- **Feature layers:** features [3] (48ch, 32x32) + features [5] (112ch, 16x16) → 160-dim patch descriptor
- **Memory bank:** Coreset ratio 0.05 → 2,500 vectors from ~50,000 patches (2.2 MB)
- **Image score:** Maximum patch distance

7.3 Results

AUROC	AUPRC	F1	TN	FP	FN	TP
0.7744	0.2656	0.3755	2,578	682	143	248

Achieved with **zero training** - no gradient updates, no hyperparameter search on this backbone, no exposure to crack images.

7.4 Analysis

PatchCore achieves the best fully unsupervised result, outperforming every SVDD and AE variant by a large margin. The preprocessing ablation reveals something interesting: raw images (no preprocessing) outperform CLAHE (0.785 vs 0.765 val AUROC), the opposite of what was found in SVDD. EfficientNet-B4's features are already sensitive enough to crack texture without contrast enhancement, the backbone does the heavy lifting that preprocessing was compensating for in weaker models.

The remaining gap from perfect (AUROC 0.77 vs 1.0) is largely attributable to the inherent ambiguity of rough-but-normal concrete texture, 682 normal patches are genuinely far from the typical normal distribution. This is a dataset-level challenge, not a method limitation.

This raises the key question for the rest of the project: PatchCore's 0.77 ceiling represents the best achievable performance from normal-only information. Any improvement must come from crack-side knowledge.

---

8. Student-Teacher Distillation

8.1 Concept

Two networks share the same ResNet-18 backbone architecture:

- **Teacher:** Pretrained ImageNet weights, fully frozen
- **Student:** Randomly initialised, trained to mimic the teacher's feature maps on normal images only

On normal images, the student learns to replicate the teacher closely. On crack images, unseen during training, the student fails, producing high MSE divergence as the anomaly score.

The **critical design choice** is random student initialisation. Using pretrained weights for the student fails because the student already knows how to mimic the teacher from day one, the distillation loss starts near-zero, and no anomaly signal develops. Random initialisation forces genuine learning: the initial loss (0.085) is ~20x higher than the pretrained-init version (0.004), confirming the student builds a real divergence gap.

8.2 Variants and Results

Variant	Feature Layers	AUROC	AUPRC	F1	TN	FP	FN	TP
A, layer3	256-dim, stride 16	0.7114	0.2258	0.2959	2,626	634	213	178
B, FPN (layer2+3)	Multi-scale, normalised	0.6735	0.1928	0.2583	2,815	445	267	124
C, layer4	**512-dim, stride 32**	**0.7674**	**0.2909**	**0.3548**	**2,654**	**606**	**176**	**215**
D, layer3+Head	layer3 + MLP projection	0.7188	0.2110	0.2931	2,399	861	176	215
E, layer4+Head	layer4 + MLP projection	0.7628	0.2952	0.3720	2,802	458	197	215

Layer4 (Variant C) is best. Layer4 features (512-dim, stride 32) encode the deepest semantic structure, abstract representations that require the most training for a random student to replicate. Crack patches remain genuinely out-of-distribution at this level, producing the widest divergence gap.

FPN (Variant B) hurts. Layer2 features (edges, colours, low-level texture) are too easy for a random student to replicate generically across both normal and crack patches, diluting the layer3 anomaly signal. Adding lower-level features adds noise, not information.

Projection head (Variants D, E): Gives the student extra capacity but trades recall for precision, fewer false positives but more missed cracks. Net effect is neutral at best.

This finding is consistent across the entire project: **layer4 > layer3 > layer2** for anomaly detection on SDNET2018. Higher abstraction levels encode more semantically meaningful differences between normal and cracked concrete.

8.3 PatchCore vs Student-Teacher Comparison

Metric	PatchCore (EB4)	S-T layer4
AUROC	**0.7744**	0.7674
AUPRC	0.2656	**0.2909**
FP	682	**606**
FN	**143**	176
TP	**248**	215

The two methods are essentially equivalent (~0.77 AUROC). PatchCore has a slight recall advantage; S-T has a slight precision advantage. The meaningful distinction is architectural: PatchCore requires no training and generalises from memorised examples, while S-T is a trained model that generalises from learned weights.

---

9. ProtoNet - Prototypical Network with Triplet Loss

9.1 Concept and Motivation

ProtoNet addresses PatchCore's core limitation: the memory bank has no concept of what a crack looks like hence the results plateau at around 0.77. By introducing crack labels during training via triplet loss, the model learns an embedding space that **explicitly separates** normal from anomalous images unlike SVDD which just compresses normal images into a sphere, or PatchCore which just memorises normal patches.

Crucially, the triplet objective cannot be satisfied by a trivial constant mapping unlike SVDD, collapse is geometrically impossible because the loss explicitly requires separation between the two classes:

L = relu(d (anchor, positive) – d (anchor, negative) + margin)

where anchor and positive are normal images, and negative is a crack image.

9.2 Architecture

- **Backbone:** ResNet-18 (ImageNet pretrained)
- **Freeze strategy:** Layers 1–3 frozen, layer4 fine-tuned (selected via ablation)
- **Projection head:** Linear (512→256, bias=False) → ReLU → Linear (256→256, bias=False)
- **Embedding normalisation:** L2-normalised onto the unit sphere

L2 normalisation is **correct here** (unlike SVDD where it causes collapse) because the distance metric is angular, both prototype and queries live on the same sphere, and the triplet objective explicitly maintains meaningful angular separation.

9.3 Inference

1. **Prototype construction:** Compute the mean embedding of a small support set of N normal images → single prototype vector on the unit sphere
2. **Anomaly score:** L2 distance from query embedding to prototype, higher = more anomalous
3. **Threshold:** τ calibrated on validation set

9.4 ProtoNet Baseline (Notebook 4)

Training configuration: Adam (lr=3e-4, wd=1e-4), CosineAnnealingLR (T_max=40), batch=32 triplets, margin=0.5, CLAHE preprocessing, patience=7.

Epoch	Loss	Active Triplet Fraction	Val AUROC
1	0.0269	10.13%	**0.7872**
5	0.0064	2.60%	0.7408
8	—	—	Early stop

Val AUROC peaks at epoch 1 (0.7872) and declines, the model overfits to the triplet geometry quickly. Early stopping at epoch 8 restores the best weights.

Baseline test results:

AUROC	AUPRC	F1	TN	FP	FN	TP
0.7673	0.4876	0.5131	3,170	90	225	166

ProtoNet's AUPRC of **0.4876 is nearly 2x PatchCore's 0.2656**, and it produces only 90 false positives vs PatchCore's 682, a 7x improvement in precision. In a real structural inspection setting, false positives translate to unnecessary re-inspections and wasted resources.

---

10. ProtoNet Ablation Study

All ablation experiments used 2,000 triplets/epoch for speed. Each hyperparameter was tested independently with all others held at the baseline configuration.

10.1 Embedding Dimension

emb_dim	val AUROC	test AUROC	F1
64	0.6886	0.7216	0.5082
128	0.7008	0.6925	0.5055
256	**0.7071**	**0.7025**	**0.4437**

Larger embeddings improve separability up to 256; 512 overfits the small support set — too many dimensions to anchor reliably from 10 support images.

10.2 Triplet Margin

margin	val AUROC	test AUROC	F1
0.3	0.7394	0.7541	0.4772
0.5	**0.7878**	**0.7990**	**0.5718**
0.7	0.7216	0.7054	0.4821
1.0	0.7246	0.7024	0.4457

Margin 0.5 provides the right balance: large enough to enforce meaningful separation but small enough that the constraint doesn't push crack embeddings so far that normal-looking cracks fall inside the normal cluster.

10.3 Freeze Strategy

freeze_until	val AUROC	test AUROC	F1
layer1	0.7577	0.7484	0.5168
layer2	0.7882	0.7958	0.5917
layer3	**0.7953**	**0.7827**	**0.5585**

Consistent with findings across the entire project, layer4 features are most discriminative for this task. Freezing through layer3 and fine-tuning only layer4 preserves enough ImageNet structure while allowing task-specific adaptation.

10.4 Preprocessing

Preprocessing	val AUROC	test AUROC	F1
none	0.7379	0.7142	0.4547
clahe	**0.7582**	**0.7730**	**0.5123**

CLAHE consistently improves crack visibility by enhancing local contrast, leading to better triplet separation. This contrasts with PatchCore where raw images were better, ProtoNet's fine-tuned encoder benefits from contrast enhancement that EfficientNet-B4's frozen features do not need.

10.5 Support Set Size

n_support	val AUROC	test AUROC	F1
1	0.7743	0.7694	0.5168
5	0.7841	0.7691	0.5123
10	**0.7976**	**0.7762**	**0.5168**
20	0.6927	0.6706	0.5031
50	0.7217	0.7078	0.5061
100	0.7225	0.7082	0.5083

Performance peaks at 10 support images. Larger support sets introduce normal-image variance that diffuses the prototype averaging too many images smooths out discriminative features. This is a practical advantage: **only 10 labelled normal images are needed at inference time**.

---

11. ProtoNet Final Model

Using the best hyperparameters from ablation, we trained the final model with the full **15,000 triplets/epoch** budget (vs 2,000 in ablation) and relaxed early stopping (MAX_EPOCHS=40, patience=7).

Best hyperparameters: *emb_dim=256, margin=0.5, freeze=layer3, preprocessing=clahe, n_support=10*

11.1 Training

Epoch	Loss	Active Triplets	Val AUROC
1	0.124687	29.42%	0.7802
4	0.004968	2.37%	**0.8337**
11 (Early stop)	0.002263	0.66%	0.8036

Best val AUROC: **0.8337** (epoch 4). Early stopping at epoch 11.

11.2 Final Test Results

AUROC	AUPRC	F1	TN	FP	FN	TP
0.8049	**0.6156**	**0.6065**	3,203	57	196	195

To reproduce: run `10_protonet_final.ipynb` cells 1–5, then Section 7.

---

12. Binary CNN - Supervised Upper Bound

12.1 Concept

The Binary CNN is **not an anomaly detection model**. It is fully supervised, using labelled examples of both normal and crack patches during training with BCEWithLogitsLoss. It serves as a performance ceiling: the best achievable result if labelled crack data were available at training time. The gap between this and anomaly detection methods quantifies the cost of not having crack labels.

12.2 Architecture

- Backbone: ResNet-18 (ImageNet pretrained, **fully fine-tuned** no freezing)
- Head: FC (512→1) + Sigmoid
- Loss: BCEWithLogitsLoss with *pos_weight=8.33* (inverse class frequency) to handle imbalance
- Augmentation: Random horizontal/vertical flip, rotation ±15°, colour jitter

12.3 Hyperparameter Search

Config	LR	Freeze	Val AUROC
1	1e-3	layer2	0.9550
2	1e-4	layer2	0.9611
5	1e-4	layer1	0.9632
6	1e-4	layer3	0.9532
7	**1e-4**	**None**	**0.9640**

Freezing any layers consistently hurts, full fine-tuning is required to adapt ImageNet features to concrete crack texture.

12.4 Final Test Results

AUROC	AUPRC	F1	TN	FP	FN	TP
0.9649	0.8774	0.7973	3,200	60	92	299

Conclusion: The Binary CNN achieves 0.9649 AUROC, a dramatic jump from every anomaly detection method. Crack detection on SDNET2018 is not an inherently hard perceptual problem given sufficient labelled data, a standard ResNet-18 classifier solves it with high confidence. The bottleneck in anomaly detection is the **absence of crack-side supervision**, not the architecture or training procedure.

13. Results & Comparison

13.1 Full Model Comparison

Model	Type	AUROC	AUPRC	F1	Notebook
Binary CNN (supervised)	Supervised upper bound	**0.9649**	**0.8774**	**0.7973**	*6_binary_cnn.ipynb*
ProtoNet Final	**Semi-supervised**	**0.8049**	**0.6156**	**0.6065**	*10_protonet_final.ipynb*
ProtoNet ablation best	Semi-supervised	0.8213	0.6406	0.6069	*9_protonet_ablation.ipynb*
PatchCore EB4 (no-train)	Unsupervised	0.7744	0.2656	0.3755	*2_patchcore_efficientnet.ipynb*
S-T layer4	Unsupervised	0.7674	0.2909	0.3548	*8_student_teacher.ipynb*
ProtoNet baseline	Semi-supervised	0.7673	0.4876	0.5131	*4_few_shot.ipynb*
Patch SVDD (ResNet-18)	Unsupervised	0.7067	0.2178	0.3127	*7_patch_svdd.ipynb*
DeepSVDD AE warm-start	Unsupervised	0.6434	0.1855	0.2542	*1_deep_svdd_with_F_norm.ipynb*
ConvAE Baseline	Unsupervised	0.6142	0.1497	0.2266	*2_autoencoder.ipynb*
Deep SVDD + F.normalize	Unsupervised	0.5781	0.1287	0.2077	*1_deep_svdd_with_F_norm.ipynb*

13.2 Key Observations

AUROC vs AUPRC gap: PatchCore (AUROC 0.7744) has an AUPRC of only 0.2656, indicating many false positives at low thresholds, only 2.5x better than random (0.107). ProtoNet Final's AUPRC of 0.6156 is 5.8x better than random, reflecting far better precision in identifying the minority crack class.

False positive comparison: ProtoNet Final produces only **57 false positives** vs PatchCore's 682. In a real inspection workflow, false positives trigger unnecessary re-inspections, 12x fewer false alarms makes ProtoNet dramatically more practical.

Semi-supervision advantage: ProtoNet uses crack supervision only during training (not at inference, where only normal support images are needed). This minimal supervision, 1,826 crack examples alongside 15,207 normal ones, produces a 0.63 AUPRC improvement over the best fully-unsupervised method.

Architecture hierarchy: Reconstruction-based methods (ConvAE, SVDD) underperform due to over-reconstruction and collapse. Feature comparison methods (PatchCore, S-T) are more stable. Distance-to-prototype on the unit sphere (ProtoNet) is the most effective formulation.

Cross-method agreement on layer4: Every method that tested feature level consistently found layer4 > layer3 > layer2. Abstract semantic features encode more meaningful differences between normal and cracked concrete than low-level texture features.

---

14. Failure Analysis

14.1 False Negatives - Missed Cracks

ProtoNet Final: **196 false negatives** out of 391 crack images (50% miss rate).

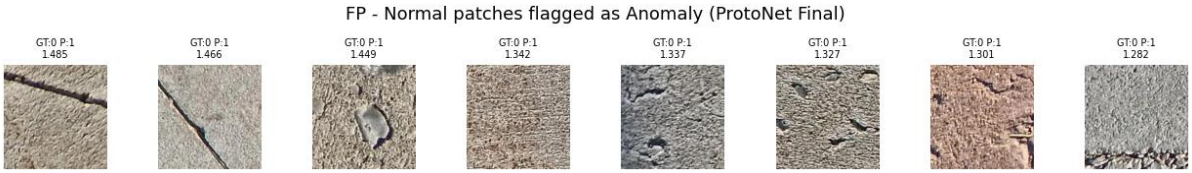
Primary cause - Hairline cracks: Shallow, narrow cracks whose visual texture closely resembles clean pavement. Even with CLAHE preprocessing enhancing local contrast, hairline cracks at certain orientations produce embeddings that fall within the normal prototype radius. Some of these hairline cracks are also in between coarse aggregate or weathered surfaces as well.

Secondary cause - Low-contrast cracks: Cracks in light-coloured pavement where the luminance difference between crack and surface is minimal. The encoder cannot distinguish these from normal surface texture variation.

To reproduce failure cases: run the manual test cell in `9_protonet_ablation.ipynb`, filtering for crack images:

```
crack_indices = [i for i, r in enumerate(test_records) if r['label'] == 1]
idx = random.choice(crack_indices)
```

14.2 False Positives — Normal Flagged as Crack



ProtoNet Final: **57 false positives** out of 3,260 normal images (1.7% false alarm rate).

Primary cause - Surface stains and oil marks: Dark linear patterns from oil or water stains create features similar to crack texture, pushing the embedding further from the normal prototype.

Secondary cause - Rough aggregate texture: Some pavement samples with particularly coarse aggregate or weathered surfaces have anomalous-looking texture that the encoder maps outside the normal cluster.

14.3 SVDD Systematic Failure

Deep SVDD's AUROC of 0.6434 indicates weak but non-trivial discriminability. The poor confusion matrix reflects an unfavourable threshold choice combined with the model's tendency to compress scores into a narrow range, a symptom of hypersphere collapse, where c attracts all representations rather than separating classes.

14.4 Autoencoder Over-Reconstruction

After ~10 epochs, ConvAE's reconstruction MSE for crack images drops below that of some normal images, **inverting the anomaly signal**. This is because the decoder's capacity (256-dim bottleneck, 4 transpose conv blocks) is sufficient to generalise reconstruction to crack textures it never saw in training. The score distributions overlap so heavily that no threshold provides useful discrimination.

---

15. Reproducibility Guide

Every figure in this report corresponds to a specific notebook cell:

Figure	Notebook	Section
Training loss & val AUROC curves	*10_protonet_final.ipynb*	Section 8
Score distribution histogram	*10_protonet_final.ipynb*	Section 9, cell 1

Figure	Notebook	Section
ROC curve	*10_protonet_final.ipynb*	Section 9, cell 2
PR curve	*10_protonet_final.ipynb*	Section 9, cell 3
Confusion matrix	*10_protonet_final.ipynb*	Section 9, cell 4
Full model comparison table	*10_protonet_final.ipynb*	Section 10
ProtoNet ablation plots	*9_protonet_ablation.ipynb*	Final summary cells
S-T variant comparison	*8_student_teacher.ipynb*	Final comparison cell
SVDD AvgPool vs MaxPool curves	*1_deep_svdd_with_F_norm.ipynb*	Iteration 3 cell
AE variant comparison	*2_autoencoder.ipynb*	Final summary cell
Binary CNN training curves	*6_binary_cnn.ipynb*	Training cell

All plots are automatically saved to *results/* when cells are run.

Load and reproduce without retraining:

```
from src.dataset import CrackDataset, load_splits
from src.train_utils import calibrate_threshold, compute_metrics

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
ckpt = torch.load('checkpoints/proto_final.pt', map_location=DEVICE,
weights_only=True)

model = ProtoNet(emb_dim=ckpt['emb_dim'],
freeze_until=ckpt['freeze_until']).to(DEVICE)
model.load_state_dict(ckpt['model_state'], strict=False)
model.build_prototype(support_recs, ckpt['preprocessing'], DEVICE)
  # → proceed to Section 7 of 10_protonet_final.ipynb
```

---

16. Conclusion

This project explored seven deep learning architectures across 20+ variants for one-class anomaly detection on the SDNET2018 concrete crack dataset. The key findings are:

1. **Reconstruction-based methods fail systematically** - ConvAE and SVDD both converge to degenerate solutions (over-reconstruction and hypersphere collapse respectively). No architectural modification (SE, CBAM, perceptual loss, warm-start, MaxPool) was sufficient to overcome the fundamental limitations of these formulations on high-variance natural textures.

2. **Feature comparison methods plateau at ~0.77 AUROC** - both PatchCore and Student-Teacher (layer4) achieve essentially the same result despite very different approaches. This represents the ceiling of normal-only information on this dataset.

3. **Minimal crack supervision unlocks significantly better performance** - ProtoNet Final achieves AUROC 0.8049 and AUPRC 0.6156 using crack labels only during training and 10 normal support images at inference. The AUPRC improvement (+0.35 over PatchCore) demonstrates that the benefit is precision, not just discrimination.

4. **Layer4 features are universally best** - consistent across SVDD, Student-Teacher, and ProtoNet ablation. Abstract semantic features encode meaningful crack/normal differences that low-level texture features cannot.

5. **The supervised ceiling (Binary CNN, AUROC 0.9649) remains 0.16 above the best anomaly detection model** - a reasonable cost for the one-class constraint in deployment scenarios where crack labels are scarce.

Recommended deployment model: ProtoNet Final (*checkpoints/proto_final.pt*) - AUROC 0.8049, AUPRC 0.6156, F1 0.6065, only 57 false alarms per 3,260 normal images (1.7% false positive rate), suitable for automated first-pass screening.

References

Ruff, L., et al. (2018). Deep One-Class Classification. *Proceedings of ICML 2018*.

Roth, K., et al. (2022). Towards Total Recall in Industrial Anomaly Detection (PatchCore). *CVPR 2022*.

Yi, J. & Yoon, S. (2021). Patch SVDD: Patch-level SVDD for Anomaly Detection and Segmentation. *ACCV 2021*.

Ren, M., et al. (2024). An annotated street view image dataset for automated road damage detection (SVRDD). *Scientific Data*, 11, 407. https://doi.org/10.1038/s41597-024-03263-7

Maeda, H., et al. (2020). Road Damage Detection Dataset (RDD2020). *IEEE BigData 2020*.

Arya, D., et al. (2022). RDD2022: A multi-national image dataset for automatic road damage detection. *arXiv:2209.08538*.

Freitas, et al. (2025). Preprocessing filters for road defect detection. *Reference results cited in Section 2.3*.

Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-Excitation Networks. *CVPR 2018*.

Woo, S., et al. (2018). CBAM: Convolutional Block Attention Module. *ECCV 2018*.